4.1.1. Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the `groupby` and `mean()` operations.

**Code:**

```
import pandas as pd

numbers = list(map(int, input().split()))

# Create a Pandas series from the list of numbers

series=pd.Series(numbers)

# Grouping by even and odd numbers and calculating the mean

grouped = series.groupby(series%2==0).mean()

# Display the mean of even and odd numbers with labels

grouped.index = ['Even' if is_even else 'Odd' for is_even in grouped.index]

print("Mean of even and odd numbers:")

print(grouped)
```

4.1.2. A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

**Create the DataFrame:**

- Convert the dictionary to a Pandas DataFrame.

**Add a new row:**

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

**Modify a row:**

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

**Delete a row:**

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row.

**Add a new column:**

- Add a column **Gender** with values taken from the user.
- Display the DataFrame after adding the new column.

**Modify a column:**

- Convert names to uppercase.
- Display the DataFrame after modifying the column.

**Delete a column:**

- Remove the **Age** column.
- Display the DataFrame after deleting the column.

**Code:**

```
import pandas as pd

# Provided dictionary of lists

data = {

    'Name': ['Alice', 'Bob', 'Charlie'],

    'Age': [25, 30, 35],
```

```
}
# Convert the dictionary to a DataFrame
df = pd.DataFrame(data)
# Display the original DataFrame
print("Original DataFrame:")
print(df)
# Adding a new row
new_name = input("New name: ")
new_age = int(input("New age: "))
new_row = {'Name': new_name,'Age': new_age}
df = df.append(new_row,ignore_index = True)
# Display the DataFrame after adding a new row
print("After adding a row:\n",df)
# Modifying a row
modify_index = int(input("Index of row to modify: "))
new_age = int(input("New age: "))
df.at[modify_index,'Age'] = new_age
# Display the DataFrame after modifying a row
print("After modifying a row:")
print(df)
# Deleting a row
del_index = int(input("Index of row to delete: "))
df = df.drop(del_index).reset_index(drop = True)
# Display the DataFrame after deleting a row
```

```
print("After deleting a row:")

print(df)

# Adding a new column

genders = input("Enter genders separated by space: ").split()

df["Gender"] = genders

# Display the DataFrame after adding a new column

print("After adding a new column:")

print(df)

# Modifying a column

df['Name'] = df['Name'].str.upper()

# Display the DataFrame after modifying a column

print("After modifying a column:")

print(df)

# Deleting a column

df = df.drop(columns = 'Age').reset_index(drop = True)

# Display the DataFrame after deleting a column

print("After deleting a column:")

print(df)
```

### 4.1.3. Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

- Display the first five rows of the data frame.
- Calculate the average age of the students (limit the average age up to 2 decimal places).

- Filter out the students who have a grade above a certain threshold (consider the threshold grade is **'B'**).

**Code:**

```
import pandas as pd

# Read the text file into a DataFrame

file = input()

data = pd.read_csv(file, sep="\s+", header=None, names=["Name", "Age", "Grade"])

# write your code here..

print("First five rows:")

print(data.head())

average_age = data['Age'].mean()

print(f"Average age: {round(average_age,2)}")

filtered_students = data[data['Grade'] <= 'B']

print("Students with a grade up to B")

print(filtered_students)
```

4.2.1. Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: `Date`, `Product`, `Quantity`, `Price`, and `City`.
- Group the data by Month and calculate the total sales for each month.
- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

**Code:**

```
import pandas as pd

# Prompt the user for the file name
```

```
file_name = input()

# Load the data

df = pd.read_csv(file_name)

df['Total Sales'] = df['Quantity'] * df['Price']

df['Date'] = pd.to_datetime(df['Date'])

df['Month'] = df['Date'].dt.to_period('M')

monthly_sales = df.groupby('Month')['Total Sales'].sum()

# Find the month with the highest total sales

best_month = monthly_sales.idxmax()

highest_sales = monthly_sales.max()

print(f"Best month: {best_month}")

print(f"Total sales: ${highest_sales:.2f}")
```

4.2.2. Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: `Date, Product, Quantity, Price,` and `City`.
- Find the product that sold the most in terms of quantity sold.
- Display the product that sold the most and the total quantity sold for that product.

**Code:**

```
import pandas as pd

# Prompt the user for the file name

file_name = input()

# Load the data

df = pd.read_csv(file_name)
```

```
product_quantity = df.groupby('Product')['Quantity'].sum()

# Find the product with the highest total quantity sold

best_product =product_quantity.idxmax()

highest_quantity = product_quantity.max()

# Display the result

print(f"Best selling product: {best_product}")

print(f"Total quantity sold: {highest_quantity}")
```

### 4.2.3. Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: `Date, Product, Quantity, Price,` and `City`.
- Group the data by `City` and calculate the total quantity of products sold for each city.
- Find the city that sold the most products (based on the total quantity sold).

**Code:**

```
import pandas as pd

# Prompt the user for the file name

file_name = input()

# Load the data

df = pd.read_csv(file_name)

# write the code..

city_sales = df.groupby('City')['Quantity'].sum()

best_city = city_sales.idxmax()

# Display the result

print(f"City sold the most products: {best_city}")
```

4.2.4. Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the following columns: Date, Product, Quantity, Price, and City.
- For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
- Output the product pair/s that was sold most frequently.

**Code:**

```
import pandas as pd

from itertools import combinations

from collections import Counter

# Prompt user to input the file name

file_name = input()

# Read data from the specified CSV file

df = pd.read_csv(file_name)

# write the code

daiy_transactions = df.groupby('Date')['Product'].apply(list)

pair_counts = Counter()

for products in daiy_transactions:

        products = sorted(products)

        pairs = list(combinations(products,2))

        pair_counts.update(pairs)

if pair_counts:

        max_count = max(pair_counts.values())

        for pair,count in pair_counts.items():
```

```
            if count == max_count:
                print(f"{pair[0]} and {pair[1]}: {count} times")
else:
    print("No product pairs found.")
```

4.2.5. You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

1. Display the first 5 rows of the dataset.
2. Display the last 5 rows of the dataset.
3. Get the shape of the dataset (number of rows and columns).
4. Get a summary of the dataset (using `.info()`).
5. Get basic statistics (mean, standard deviation, etc.) of the dataset using `.describe()`.
6. Check for missing values and display the count of missing values for each column.
7. Fill missing values in the 'Age' column with the median age.
8. Fill missing values in the 'Embarked' column with the most frequent value (`mode()`).
9. Drop the 'Cabin' column due to many missing values.
10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below,

| PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare | Cabin | Em |
|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | | | | | | | | | |

**Code:**

```
import pandas as pd

import numpy as np
```

```
# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
# 1. Display the first 5 rows of the dataset
print(data.head())
# 2. Display the last 5 rows of the dataset
print(data.tail())
# 3. Get the shape of the dataset
print(data.shape)
# 4. Get a summary of the dataset (info)
print(data.info())
# 5. Get basic statistics of the dataset
print(data.describe())
# 6. Check for missing values
print(data.isnull().sum())
# 7. Fill missing values in the ‘Age’ column with the median age
data['Age'].fillna(data['Age'].median(),inplace = True)
# 8. Fill missing values in the ‘Embarked’ column with the mode
data['Embarked'].fillna(data['Embarked'].mode()[0],inplace=True)
# 9. Drop the ‘Cabin’ column due to many missing values
data.drop(columns='Cabin',inplace = True)
# 10. Create a new column 'FamilySize’ by adding ‘SibSp' and ‘Parch'
data["FamilySize"] = data['SibSp']+data['Parch']
```

4.2.6. You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column ‘IsAlone’ which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
2. Convert the ‘Sex' column to numeric values (male: 0, female: 1).
3. One-hot encode the ‘Embarked’ column, dropping the first category.
4. Get the mean age of passengers.
5. Get the median fare of passengers.
6. Get the number of passengers by class.
7. Get the number of passengers by gender.
8. Get the number of passengers by survival status.
9. Calculate the survival rate of passengers.
10. Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below,

| PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare | Cabin | Em |
|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | | | | | | | | | |

**Code:**

```
import pandas as pd

import numpy as np

# Load the Titanic dataset

data = pd.read_csv('Titanic-Dataset.csv')

data['FamilySize'] = data['SibSp'] + data['Parch']

# 1. Create a new column ‘IsAlone' (1 if alone, 0 otherwise)

data['IsAlone']=np.where(data['FamilySize'] == 0,1,0)

# 2. Convert ‘Sex' to numeric (male: 0, female: 1)

data['Sex'] = data['Sex'].map({'male':0,'female':1})

# 3. One-hot encode the ‘Embarked' column
```

```
data = pd.get_dummies(data, columns=['Embarked'])

# 4. Get the mean age of passengers

mean_age = data['Age'].mean()

print(mean_age)

# 5. Get the median fare of passengers

median_fare = data['Fare'].median()

print(median_fare)

# 6. Get the number of passengers by class

print( data['Pclass'].value_counts())

# 7. Get the number of passengers by gender

print(data['Sex'].value_counts())

# 8. Get the number of passengers by survival status

print(data['Survived'].value_counts())

# 9. Calculate the survival rate

survival_rate = data['Survived'].mean()

print(format(survival_rate))

# 10. Calculate the survival rate by gender

survival_by_gender = data.groupby('Sex')['Survived'].mean()

print( survival_by_gender)
```

4.2.7. You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.
2. Calculate the survival rate by embarkation location (Embarked_S).

3. Calculate the survival rate by family size (FamilySize).
4. Calculate the survival rate by being alone (IsAlone).
5. Get the average fare by passenger class (Pclass).
6. Get the average age by passenger class (Pclass).
7. Get the average age by survival status (Survived).
8. Get the average fare by survival status (Survived).
9. Get the number of survivors by class (Pclass).
10. Get the number of non-survivors by class (Pclass).

The Titanic dataset contains columns as shown below,

| PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare | Cabin | Em |
|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | | | | | | | | | |

**Code:**

```
import pandas as pd

import numpy as np

# Load the Titanic dataset

data = pd.read_csv('Titanic-Dataset.csv')

data['FamilySize'] = data['SibSp'] + data['Parch']

data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)

data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# 1. Calculate the survival rate by class

print(data.groupby('Pclass')['Survived'].mean())

# 2. Calculate the survival rate by embarked location

print(data.groupby('Embarked_S')['Survived'].mean())

# 3. Calculate the survival rate by family size

print(data.groupby('FamilySize')['Survived'].mean())

# 4. Calculate the survival rate by being alone
```

```
print(data.groupby('IsAlone')['Survived'].mean())

# 5. Get the average fare by class

print(data.groupby('Pclass')['Fare'].mean())

# 6. Get the average age by class

print(data.groupby('Pclass')['Age'].mean())

# 7. Get the average age by survival status

print(data.groupby('Survived')['Age'].mean())

# 8. Get the average fare by survival status

print(data.groupby('Survived')['Fare'].mean())

# 9. Get the number of survivors by class

print(data[data['Survived'] == 1]['Pclass'].value_counts())

# 10. Get the number of non-survivors by class

print(data[data['Survived'] == 0]['Pclass'].value_counts())
```

4.2.8. You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Get the number of survivors by gender (Sex).
2. Get the number of non-survivors by gender (Sex).
3. Get the number of survivors by embarkation location (Embarked_S).
4. Get the number of non-survivors by embarkation location (Embarked_S).
5. Calculate the percentage of children (Age < 18) who survived.
6. Calculate the percentage of adults (Age >= 18) who survived.
7. Get the median age of survivors.
8. Get the median age of non-survivors.
9. Get the median fare of survivors.
10. Get the median fare of non-survivors.

The Titanic dataset contains columns as shown below,

| PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare | Cabin | Em |
|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | | | | | | | | | |

**Code:**

```
import pandas as pd

import numpy as np

# Load the Titanic dataset

data = pd.read_csv('Titanic-Dataset.csv')

data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# 1. Get the number of survivors by gender

survivors_by_gender=data[data['Survived']==1]['Sex'].value_counts()

print(survivors_by_gender)

# 2. Get the number of non-survivors by gender

print(data[data['Survived']==0]['Sex'].value_counts())

# 3. Get the number of survivors by embarked location

print(data[data['Survived']==1]['Embarked_S'].value_counts())

# 4. Get the number of non-survivors by embarked location

print(data[data['Survived']==0]['Embarked_S'].value_counts())

# 5. Calculate the percentage of children (Age < 18) who survived

print(data[data['Age']<18]['Survived'].mean())

# 6. Calculate the percentage of adults (Age >= 18) who survived

print(data[data['Age']>=18]['Survived'].mean())

# 7. Get the median age of survivors

print(data[data['Survived']==1]['Age'].median())
```

```
# 8. Get the median age of non-survivors

print(data[data['Survived']==0]['Age'].median())

# 9. Get the median fare of survivors

print(data[data['Survived']==1]['Fare'].median())

# 10. Get the median fare of non-survivors

print(data[data['Survived']==0]['Fare'].median())
```